

EduOS Simulator Report

Operating Systems Assignment

1. Introduction

This project implements an educational operating system simulator called EduOS using C and Python. The simulator demonstrates major operating systems concepts including process management, threading, synchronization, deadlocks, inter-process communication, and CPU scheduling algorithms.

The purpose of the project was to bridge theoretical operating systems concepts with practical implementation. The project combines low-level systems programming in C with scheduling visualization and analysis in Python.

2. Objectives

- Simulate process management using PCB structures
 - Demonstrate threading and synchronization
 - Implement CPU scheduling algorithms
 - Demonstrate deadlocks and IPC concepts
 - Integrate C and Python modules
 - Visualize scheduling using Gantt charts and comparison charts
-

3. Design Rationale

The project was divided into two major layers: a low-level C simulator and a Python scheduling subsystem.

C was selected because it provides low-level system programming capabilities, direct memory access, and pthread support. Python was selected because it simplifies scheduling visualization and data analysis.

JSON was selected as the communication bridge between C and Python because it is lightweight, human-readable, and easy to parse. The modular structure also improved maintainability, debugging, and extensibility.

The system architecture was intentionally modular:

- C core simulator

- Python scheduler
- Controller module
- Documentation subsystem

This modular design simplified testing and debugging while improving project organization.

4. PCB and Process Management

A Process Control Block (PCB) structure was implemented to store process information including:

- PID
- Process state
- Priority
- Burst time
- Arrival time
- Memory requirements
- Thread count

The PCB acted as the simulated operating system process table.

The project implemented simplified process operations including:

- `edu_fork()` – simulated process creation
- `edu_exec()` – simulated loading programs
- `edu_exit()` – simulated process termination
- `edu_ps()` – displayed active processes

The PCB table was exported to JSON format for integration with Python scheduling modules.

```

PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\python_scheduler> cd..
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment> cd c_core
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\eduos.exe
[FORK] Process created: Chrome (PID 1)
[FORK] Process created: Spotify (PID 2)
[EXEC] PID 1 now running Firefox

PROCESS TABLE
-----
PID: 1 | NAME: Firefox | STATE: 2
PID: 2 | NAME: Spotify | STATE: 1
[EXIT] PID 2 terminated

PROCESS TABLE
-----
PID: 1 | NAME: Firefox | STATE: 2
PID: 2 | NAME: Spotify | STATE: 4
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core>

```

5. Threading and Synchronization

pthread threads were used to demonstrate concurrent execution.

A shared counter variable was intentionally accessed simultaneously by multiple threads to demonstrate race conditions.

The race-condition version produced inconsistent counter values because multiple threads modified shared memory simultaneously.

Mutex locks were later introduced to protect the critical section. After synchronization was added, the counter consistently produced correct results.

This demonstrated:

- Race conditions
- Critical sections
- Mutex synchronization
- Shared memory protection

```

PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> gcc thread_demo.c -o thread_demo -lpthread
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 100000
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 136437
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 100000
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 133168
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 115641
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core>

```

```
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 200000
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 200000
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 200000
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\thread_demo.exe
Final Counter: 200000
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> 
```

6. Deadlock Demonstration

Deadlock was demonstrated using two mutex locks acquired in opposite order by two different threads.

This caused circular waiting where each thread waited indefinitely for a resource held by the other thread.

```
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\c_core> .\deadlock_demo.exe
Thread 1 acquired lock1
Thread 2 acquired lock2
Thread 1 waiting for lock2
Thread 2 waiting for lock1

```

The demonstration illustrated the four conditions necessary for deadlock:

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

The project also discussed possible deadlock prevention techniques such as resource ordering.

7. Inter-Process Communication (IPC)

The project explored IPC concepts including pipes and shared memory.

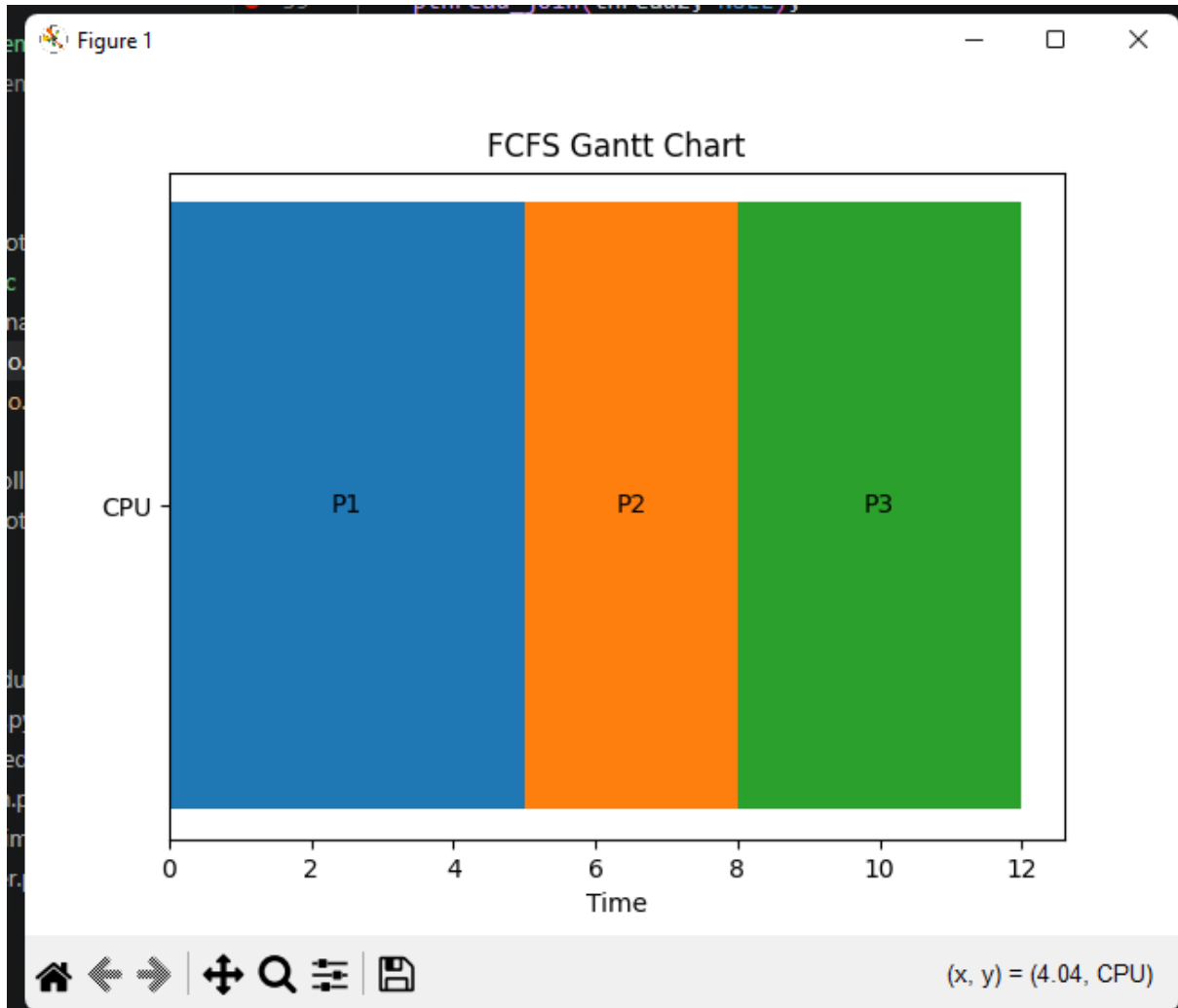
Pipes were used to illustrate sequential communication between processes, while shared memory was discussed as a faster communication mechanism using common memory regions.

Due to Windows POSIX compatibility limitations, some IPC demonstrations were discussed theoretically rather than fully implemented.

8. CPU Scheduling Algorithms

The project implemented four CPU scheduling algorithms:

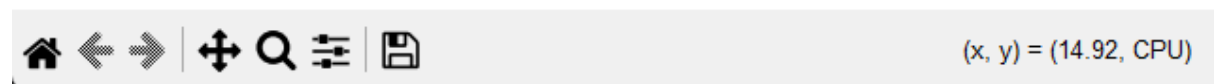
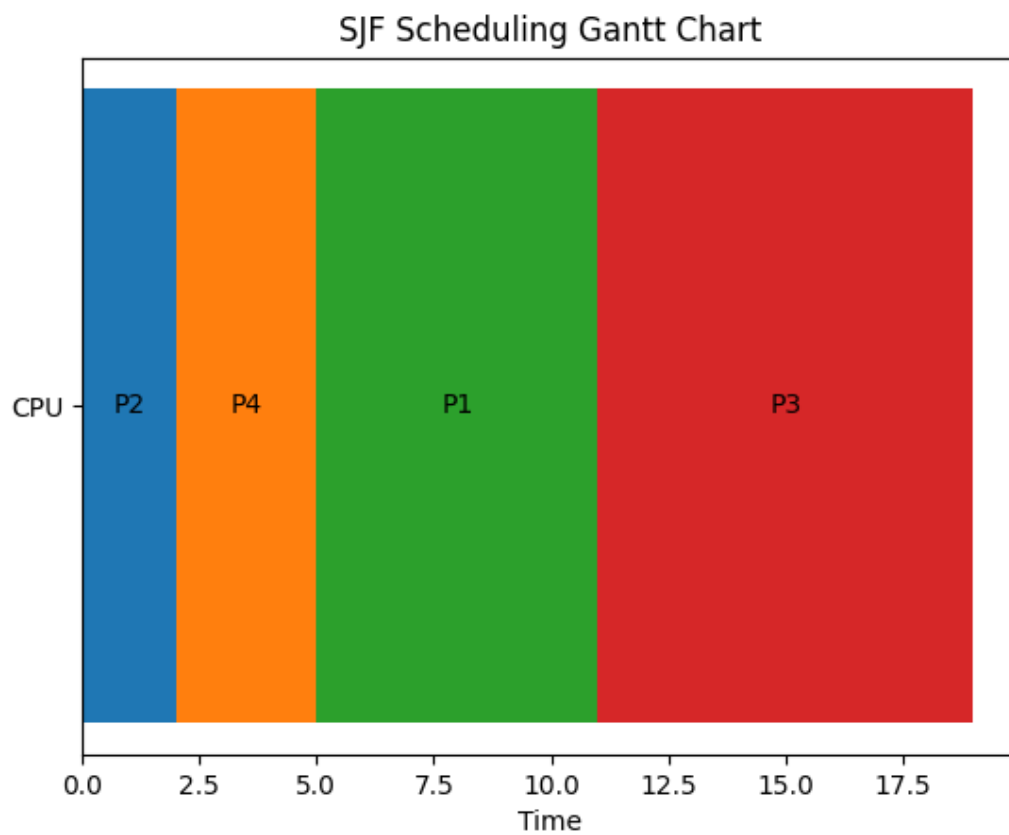
First Come First Served (FCFS)



Processes execute in arrival order. This algorithm is simple but may suffer from convoy effects.

Shortest Job First (SJF)

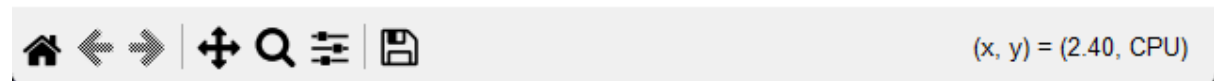
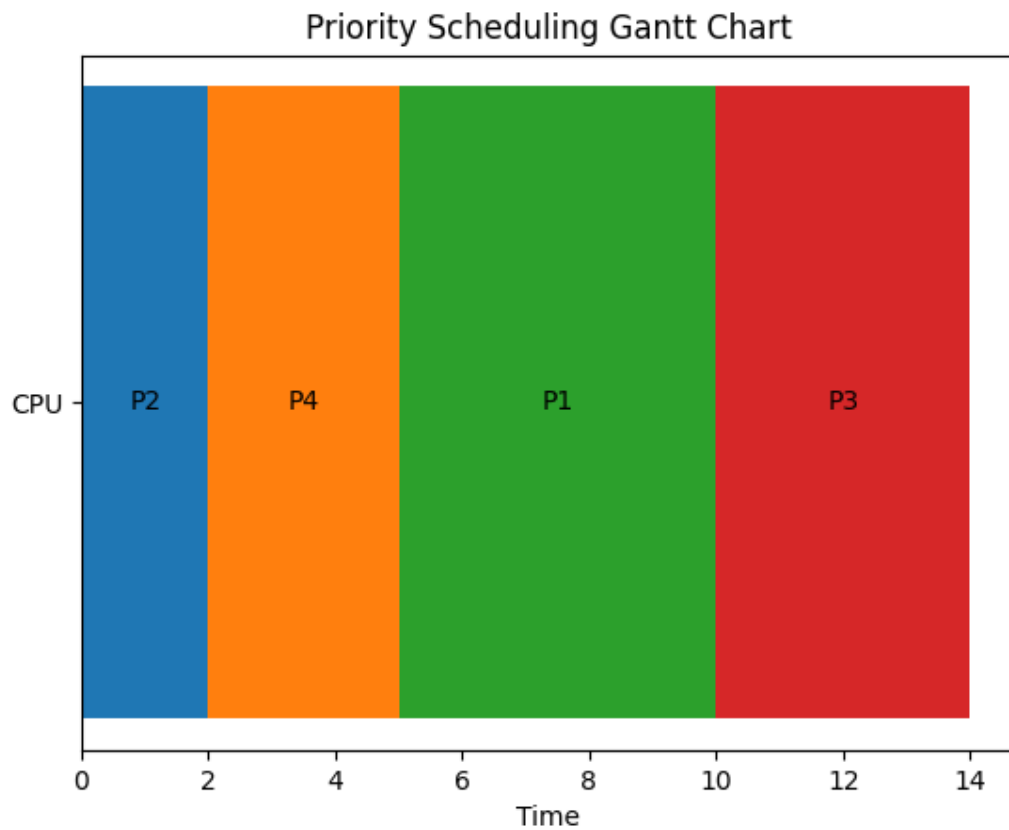
Figure 1



Processes with the shortest burst time execute first. This minimizes average waiting time but may cause starvation.

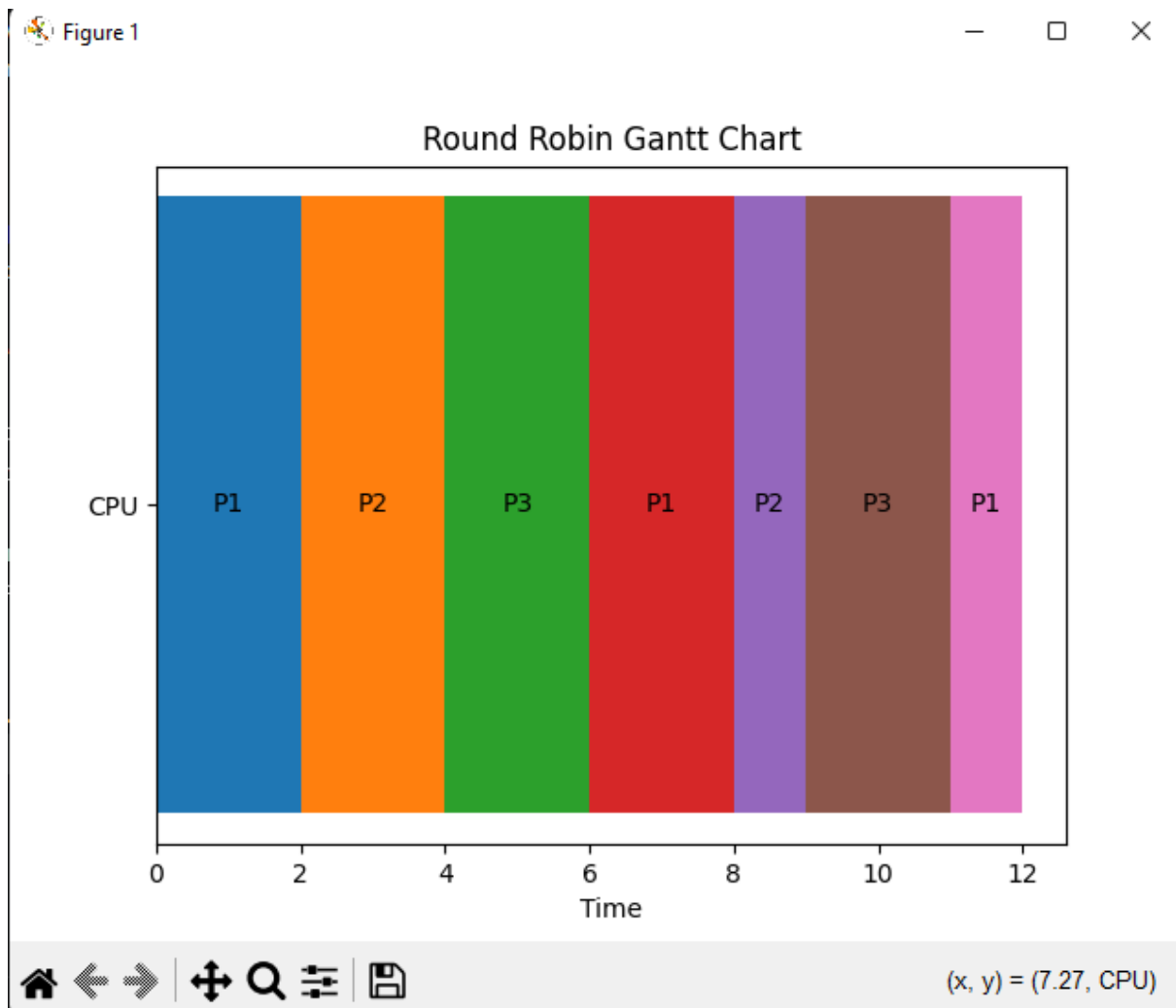
Priority Scheduling

Figure 1



Processes execute according to assigned priority levels. Higher-priority tasks execute first.

Round Robin Scheduling



Processes receive equal CPU time slices using a fixed time quantum. This improves fairness and responsiveness.

Gantt charts were generated for all scheduling algorithms using matplotlib.

Comparison charts were also generated to compare:

- Waiting time
- Turnaround time
- Fairness
- Responsiveness

9. Integration Between C and Python

```
PS C:\Users\Goodreach\Documents\GitHub\operating-systems-assignment\python_scheduler> python json_reader.py  
PROCESSES LOADED FROM C SIMULATOR  
PID: 1 | Name: Firefox | State: 2  
PID: 2 | Name: Spotify | State: 4
```


The C simulator exported process information into a JSON file.

Python scheduling modules later loaded this data for scheduling analysis and visualization.

The controller module automated the workflow by:

1. Launching the C simulator
2. Loading the generated JSON
3. Running scheduling analysis

This demonstrated interoperability between multiple programming languages.

10. Analysis of Results

The outputs demonstrated expected operating systems behavior.

FCFS scheduling was simple but suffered from convoy effects.

SJF minimized average waiting time and improved efficiency for short tasks.

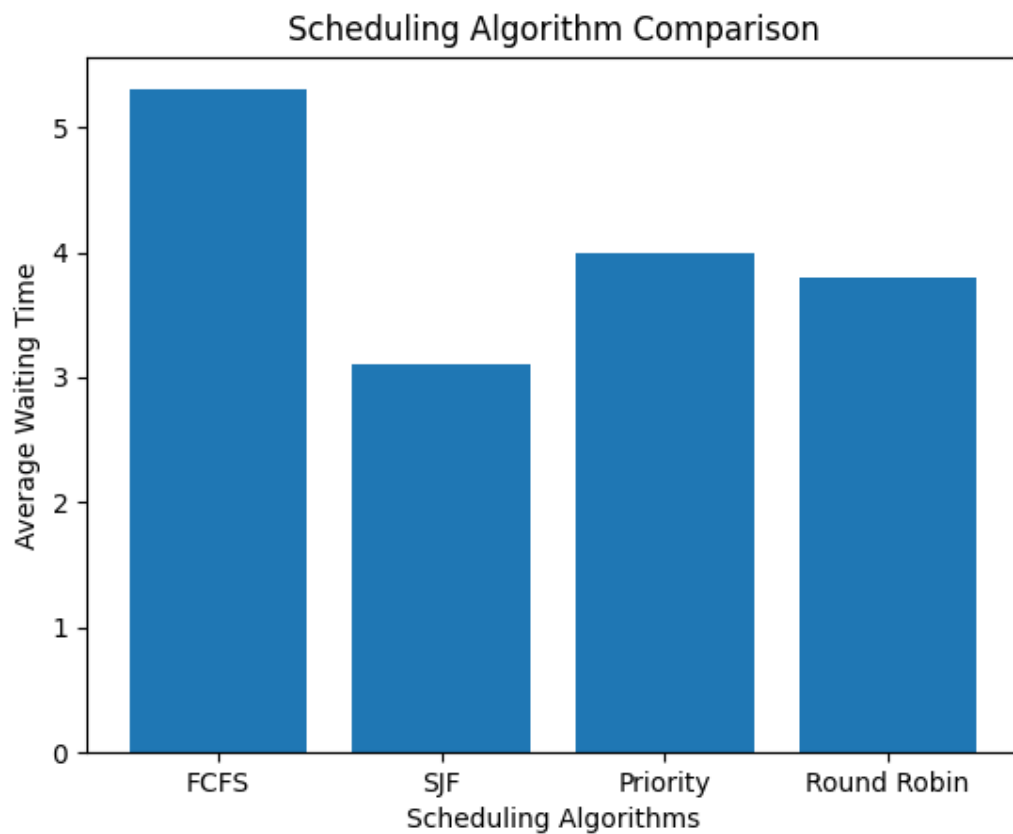
Priority Scheduling favored urgent tasks but introduced starvation risk for low-priority processes.

Round Robin improved responsiveness and fairness through time slicing.

The race-condition demonstration initially produced inconsistent results, while mutex synchronization corrected the issue completely.

The deadlock simulation successfully demonstrated indefinite waiting behavior.

The Gantt charts visually demonstrated CPU allocation differences between algorithms.



11. Screenshots and Evidence

Screenshots were collected for:

- Process table output
- Race-condition demonstration
- Mutex synchronization results
- Deadlock demonstration
- FCFS Gantt chart
- SJF Gantt chart
- Priority Scheduling Gantt chart
- Round Robin Gantt chart
- Scheduling comparison chart

More screenshots inside:

[docs/screenshots/](#)

Each screenshot demonstrates a major operating systems concept implemented in the project.

12. Challenges Faced

Several challenges were encountered during development:

- Understanding synchronization concepts
- Handling race conditions
- Managing multi-file C projects
- Integrating C and Python modules
- Windows compatibility limitations with POSIX features
- Debugging thread-related issues

These challenges improved my understanding of practical systems programming.

13. Syllabus Mapping

The EduOS simulator directly implemented major Operating Systems syllabus topics including:

- Process management
- PCB structures
- Threads and concurrency
- Synchronization and mutexes
- Deadlocks
- CPU scheduling algorithms
- Inter-process communication
- Context switching concepts
- Scheduling visualization

The project therefore provided practical implementation experience for the theoretical concepts covered during the course.

14. GitHub Summary

The project was maintained using Git and GitHub version control.

Multiple commits documented:

- Feature implementation
- Debugging progress
- Scheduling modules
- Documentation updates
- Visualization improvements

The repository contains:

- C source files
- Python scheduling modules
- Screenshots
- Report documents
- README documentation
- Scheduling visualizations

The instructor was added as a collaborator to the repository.

15. Conclusion

The EduOS simulator successfully demonstrated major operating systems concepts through practical implementation.

The project strengthened understanding of:

- Process management
- Synchronization
- Deadlocks
- Scheduling algorithms
- System integration

The integration of C and Python provided both low-level systems programming experience and high-level scheduling visualization capabilities.

Overall, the project successfully connected operating systems theory with practical software implementation.

References

1. Silberschatz, A., Galvin, P.B., & Gagne, G. Operating System Concepts.
2. Python Documentation.
3. GCC Compiler Documentation.
4. POSIX pthread Documentation.